รายงาน

วิชา Internet of Things Fundamentals and Embedded Systems

เรื่อง การพัฒนาแขนกลหุ่นยนต์สำหรับการหยิบจับวัตถุพร้อมแผงควบคุมแบบ IoT

ผู้จัดทำ

67050334 นาย ปัณณวัฒน์ เลิศโกเมนกุล

67050473 นาย รัชชานนท์ เหล่าสัมพันธ์

อาจารย์ประจำรายวิชา

รศ.ธีรวัฒน์ ประกอบผล

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา

05506271 Internet of Things Fundamentals and Embedded Systems

หลักสูตรวิทยาศาสตร์บัณฑิต

สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง (สจล.)

ประจำภาคการศึกษาที่ 2 ปีการศึกษา 256

สารบัญ

บทที่1 1

บทนำ.....1

1.1ที่มาและความสำคัญ 1

1.2วัตถุประสงค์ของการศึกษา 1

1.3ประโยชน์ที่คาดว่าจะได้รับ 2

บทที่ 2 3

เอกสารและงานวิจัยที่เกี่ยวข้อง.....3

2.1 แขนกลหุ่นยนต์ (ROBOT ARM) 3

2.2 เซอร์โวมอเตอร์ (SERVO MOTOR) 4

2.3 ไมโครคอนโทรลเลอร์ ESP32 5

2.4 การควบคุมอุปกรณ์ผ่าน WEB INTERFACE 6

2.6 ระบบบันทึกและเล่นซ้ำคำสั่ง (RECORD AND REPLAY) 7

2.7 งานวิจัยหรือโครงงานที่เกี่ยวข้อง 7

บทที่ 3 9

ขั้นตอนและวิธีการดำเนินงาน 9

บทที่ 4 34

ผลการดำเนินงาน..... 34

4.1 ภาพรวมของระบบที่พัฒนา 34

4.2 โครงสร้างของระบบที่พัฒนา 34

4.3 โหมดการทำงานของระบบ 35
4.3.1 Hardware Control Mode *35*
4.3.2 WiFi Control Mode *36*
..... *36*
4.3.3 Disabled Mode *37*
4.4 ผลการทดสอบการทำงานของระบบ 37
4.5 สรุปผลการดำเนินงาน 37
บทที่ 5 38
สรุปและอภิปรายผลการดำเนินงาน 38
5.1 สรุปผลการดำเนินงาน 38
5.2 อภิปรายผลการดำเนินงาน 39
5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต 39
บรรณานุกรม 40

บทที่1

บทนำ

1.1ที่มาและความสำคัญ

ในกระบวนการทำงานหลายประเภท เช่น การหยิบจับวัตถุ การจัดเรียงสินค้า หรือการเคลื่อนย้ายชิ้นงานในโรงงาน มักต้องใช้แรงงานมนุษย์ในการทำงานซ้ำ ๆ ซึ่งอาจก่อให้เกิดปัญหา เช่น ความล้า ความผิดพลาดจากมนุษย์ และประสิทธิภาพในการทำงานที่ไม่สม่ำเสมอ นอกจากนี้ งานบางประเภทอาจต้องการความแม่นยำหรือการควบคุมที่ต่อเนื่องซึ่งมนุษย์ทำได้ยาก

ดังนั้นจึงมีความจำเป็นในการพัฒนา แขนกลหุ่นยนต์ (RobotArm) เพื่อช่วยในการหยิบจับและเคลื่อนย้ายวัตถุโดยอัตโนมัติ ซึ่งสามารถช่วยเพิ่มความแม่นยำ ลดภาระของมนุษย์ และเพิ่มประสิทธิภาพในการทำงาน โครงงานนี้จึงมีเป้าหมายในการออกแบบและพัฒนา ระบบแขนกลหุ่นยนต์ที่สามารถควบคุมการเคลื่อนที่และการหยิบจับวัตถุได้ โดยใช้ไมโครคอนโทรลเลอร์และเซอร์โวมอเตอร์ เพื่อจำลองการทำงานของแขนกลในระบบอัตโนมัติ

1.2วัตถุประสงค์ของการศึกษา

1. เพื่อศึกษาและปฏิบัติการเขียนโปรแกรมควบคุมไมโครคอนโทรลเลอร์ ESP32-S3 ด้วยภาษา MicroPython ในการสั่งงานแขนหุ่นยนต์ทั้ง 4 แกน
2. ใช้ module joystick ใช้ในการควบคุมทิศทางการหมุนของ servo ในแต่ละแกน
3. ศึกษาว่าหลักการที่ทำให้แขนหุ่นยนต์สามารถขยับไปในทิศทางที่เรากำหนดด้วยการบังคับเองในรอบแรก และสามารถทำให้แขนหุ่นยนต์ขยับตามที่เราขยับเหมือนตอนแรกไปเรื่อยๆได้ยังไง
4. สร้างหน้า web เพื่อให้โทรศัพท์สามารถควบคุมการทำงานจากระยะไกลได้

1.3ประโยชน์ที่คาดว่าจะได้รับ

1. ได้เรียนรู้หลักการออกแบบและพัฒนาแขนกลหุ่นยนต์ (Robot Arm) รวมถึงโครงสร้างทางกลและการทำงานของระบบหุ่นยนต์
2. ได้เข้าใจการควบคุม **Servo Motor** สำหรับการเคลื่อนที่ของข้อต่อแขนกลในระบบหุ่นยนต์
3. ได้พัฒนาทักษะการเขียนโปรแกรมและการใช้งาน **Microcontroller** เพื่อควบคุมอุปกรณ์อิเล็กทรอนิกส์
4. ได้เรียนรู้การพัฒนาระบบควบคุมอุปกรณ์ผ่าน **Web Interface และ WiFi (Access Point)** สำหรับการควบคุมระยะไกล
5. สามารถนำความรู้ที่ได้ไปประยุกต์ใช้กับงานด้าน **Robotics, IoT และระบบอัตโนมัติ (Automation)** ในอนาคต

บทที่ 2

เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 แขนกลหุ่นยนต์ (Robot Arm)

แขนกลหุ่นยนต์ (Robot Arm) เป็นอุปกรณ์หุ่นยนต์ที่ถูกออกแบบมาให้มีลักษณะและการเคลื่อนไหวคล้ายกับแขนของมนุษย์ โดยมีหน้าที่หลักในการหยิบจับ เคลื่อนย้าย หรือจัดเรียงวัตถุ แขนกลมักถูกนำมาใช้ในงานอุตสาหกรรม เช่น การประกอบชิ้นส่วน การจัดเรียงสินค้า (Pick and Place) และการทำงานที่ต้องการความแม่นยำสูง

โครงสร้างของแขนกลหุ่นยนต์โดยทั่วไปประกอบด้วยส่วนสำคัญดังนี้

- **Base** เป็นฐานของแขนกล ทำหน้าที่รองรับโครงสร้างทั้งหมด
- **Link** เป็นส่วนที่เชื่อมต่อระหว่างข้อต่อต่าง ๆ
- **Joint** เป็นข้อต่อที่ทำให้แขนกลสามารถเคลื่อนที่ได้

- **End Effector** เป็นส่วนปลายของแขนกล เช่น ตัวหนีบ (Gripper) สำหรับหยิบจับวัตถุ

แขนกลหุ่นยนต์สามารถควบคุมการเคลื่อนที่ได้ด้วยมอเตอร์หลายประเภท เช่น Servo Motor หรือ Stepper Motor ซึ่งช่วยให้สามารถควบคุมตำแหน่งและทิศทางการเคลื่อนที่ได้อย่างแม่นยำ

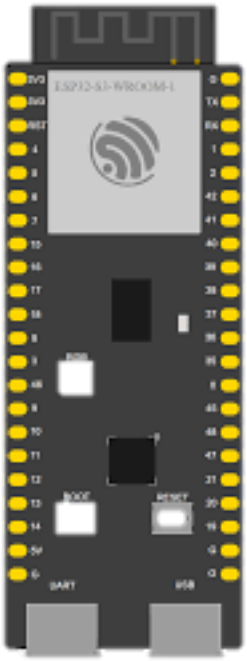
2.2 เซอร์โวมอเตอร์ (Servo Motor)

Servo Motor เป็นมอเตอร์ชนิดหนึ่งที่สามารถควบคุมตำแหน่ง ความเร็ว และทิศทางการหมุนได้อย่างแม่นยำ โดยภายในเซอร์โวมอเตอร์จะประกอบด้วยมอเตอร์ไฟฟ้า วงจรควบคุม และตัวตรวจวัดตำแหน่ง (Feedback System)

การควบคุมเซอร์โวมอเตอร์มักใช้สัญญาณ **PWM (Pulse Width Modulation)** ซึ่งเป็นสัญญาณดิจิทัลที่มีการปรับความกว้างของพัลส์เพื่อกำหนดตำแหน่งของมอเตอร์ ตัวอย่างเช่น สัญญาณ PWM ที่มีความกว้างต่างกันจะทำให้มอเตอร์หมุนไปยังตำแหน่งที่แตกต่างกัน

เซอร์โวมอเตอร์ถูกนำมาใช้อย่างแพร่หลายในงานด้านหุ่นยนต์ เนื่องจากสามารถควบคุมตำแหน่งได้อย่างแม่นยำและตอบสนองต่อคำสั่งได้อย่างรวดเร็ว

2.3 ไมโครคอนโทรลเลอร์ ESP32



ESP32 เป็นไมโครคอนโทรลเลอร์ที่พัฒนาโดยบริษัท Espressif Systems ซึ่งได้รับความนิยมอย่างมากในงานด้านระบบสมองกลฝังตัว (Embedded Systems) และ Internet of Things (IoT)

คุณสมบัติสำคัญของ ESP32 ได้แก่

- มีหน่วยประมวลผลแบบ Dual-core
- รองรับการเชื่อมต่อ **WiFi และ Bluetooth**
- มีขา GPIO สำหรับเชื่อมต่อกับอุปกรณ์ภายนอก
- รองรับการเขียนโปรแกรมด้วยภาษา Python (MicroPython) หรือ C/C++

ESP32 จึงเหมาะสำหรับการพัฒนาโครงงานที่ต้องการควบคุมอุปกรณ์อิเล็กทรอนิกส์และเชื่อมต่อเครือข่ายอินเทอร์เน็ต เช่น ระบบควบคุมหุ่นยนต์ผ่านเครือข่ายไร้สาย

2.4 การควบคุมอุปกรณ์ผ่าน Web Interface

การควบคุมอุปกรณ์ผ่าน Web Interface เป็นวิธีที่ช่วยให้ผู้ใช้งานสามารถสั่งงานอุปกรณ์ได้ผ่านเว็บเบราว์เซอร์โดยไม่จำเป็นต้องติดตั้งโปรแกรมเพิ่มเติม

ในระบบนี้ ไมโครคอนโทรลเลอร์จะทำหน้าที่เป็น **Web Server** ที่คอยรับคำสั่งจากผู้ใช้งานผ่านโปรโตคอล **HTTP (Hypertext Transfer Protocol)** เมื่อผู้ใช้กดปุ่มควบคุมบนหน้าเว็บ ระบบจะส่งคำสั่งไปยังไมโครคอนโทรลเลอร์ ซึ่งจะนำคำสั่งนั้นไปควบคุมอุปกรณ์ต่าง ๆ เช่น มอเตอร์หรือเซนเซอร์

การใช้ Web Interface ทำให้ระบบสามารถใช้งานได้สะดวกและสามารถควบคุมได้จากอุปกรณ์หลายประเภท เช่น คอมพิวเตอร์ แท็บเล็ต หรือสมาร์ตโฟน

2.5 เครือข่าย WiFi และ Access Point

https://devadiy.com/wp-content/uploads/e 1

WiFi เป็นเทคโนโลยีเครือข่ายไร้สายที่ใช้ในการเชื่อมต่ออุปกรณ์ต่าง ๆ เข้าด้วยกันผ่านคลื่นวิทยุ ในไมโครคอนโทรลเลอร์ ESP32 สามารถทำงานได้ทั้งในโหมด **Station Mode** และ **Access Point Mode**

ในโหมด **Access Point (AP Mode)** ESP32 จะทำหน้าที่เป็นจุดกระจายสัญญาณ WiFi ทำให้อุปกรณ์อื่นสามารถเชื่อมต่อเข้ามาได้โดยตรง ซึ่งเหมาะสำหรับการพัฒนาโครงงานที่ต้องการสร้างเครือข่ายเฉพาะเพื่อควบคุมอุปกรณ์ เช่น ระบบควบคุมแขนกลหุ่นยนต์ผ่านเว็บเบราว์เซอร์

2.6 ระบบบันทึกและเล่นซ้ำคำสั่ง (Record and Replay)

ระบบบันทึกและเล่นซ้ำคำสั่ง (Record and Replay) เป็นเทคนิคที่ใช้ในการบันทึกลำดับการเคลื่อนที่ของหุ่นยนต์และสามารถนำข้อมูลที่บันทึกไว้มาทำงานซ้ำได้ในภายหลัง

กระบวนการทำงานของระบบ Record and Replay มีขั้นตอนดังนี้

1. บันทึกคำสั่งหรือการเคลื่อนที่ของอุปกรณ์
2. จัดเก็บข้อมูลไว้ในหน่วยความจำหรือไฟล์
3. อ่านข้อมูลที่บันทึกไว้และส่งคำสั่งไปยังอุปกรณ์ตามลำดับเวลาเดิม

ระบบนี้ช่วยให้หุ่นยนต์สามารถทำงานซ้ำได้โดยไม่ต้องควบคุมใหม่ทุกครั้ง และสามารถนำไปใช้ในงานอุตสาหกรรมที่ต้องทำงานแบบเดิมซ้ำ ๆ

2.7 งานวิจัยหรือโครงงานที่เกี่ยวข้อง

จากการศึกษางานวิจัยและโครงงานที่เกี่ยวข้อง พบว่ามีการพัฒนาแขนกลหุ่นยนต์ในหลากหลายรูปแบบ เช่น

งานวิจัยเกี่ยวกับการพัฒนาแขนกลหุ่นยนต์ที่ใช้ **Servo Motor** ในการควบคุมการเคลื่อนที่ของข้อต่อ โดยใช้ไมโครคอนโทรลเลอร์ในการประมวลผลและควบคุมตำแหน่งของแขนกล ทำให้สามารถหยิบจับและเคลื่อนย้ายวัตถุได้อย่างแม่นยำ

นอกจากนี้ยังมีการพัฒนาโครงงานที่ใช้ **ESP32 เป็น Web Server** เพื่อควบคุมการทำงานของอุปกรณ์ผ่านเครือข่าย WiFi ผู้ใช้งานสามารถสั่งงานอุปกรณ์ผ่านหน้าเว็บได้แบบเรียลไทม์

จากการศึกษางานที่เกี่ยวข้องดังกล่าว แสดงให้เห็นว่าการนำไมโครคอนโทรลเลอร์ เซอร์โวมอเตอร์ และระบบเครือข่ายไร้สายมาประยุกต์ใช้ร่วมกันสามารถพัฒนาระบบแขนกลหุ่นยนต์ที่ควบคุมได้ผ่านเว็บและสามารถบันทึกการเคลื่อนที่เพื่อนำมาใช้งานซ้ำได้

ลำดับ	*งานวิจัย / ผู้พัฒนา*	*เทคโนโลยีที่ใช้*	*วิธีการควบคุม*	*ความสามารถของระบบ*
1	Design and Implementation of Robotic Arm using Arduino	Arduino, Servo Motor	ปุ่มควบคุม (Push Button)	ควบคุมการหมุนของแขนกลเพื่อหยิบจับวัตถุ
2	IoT Based Robotic Arm using ESP32	ESP32, WiFi, Servo Motor	Mobile Application	ควบคุมแขนกลผ่านเครือข่ายอินเทอร์เน็ต
3	Web-Based Robotic Arm Control	ESP32, Web Server, WiFi	Web Interface	ควบคุมแขนกลผ่านเว็บเบราว์เซอร์แบบเรียลไทม์
4	Robotic Arm with Motion Recording	Microcontroller, Servo Motor	Controller / Joystick	บันทึกและเล่นซ้ำการเคลื่อนที่ของแขนกล
5	**โครงงานนี้: Robot Arm Control System**	**ESP32, Servo Motor, WiFi, Web Interface, Joystick**	**Joystick และ Web Browser**	**ควบคุมแขนกลผ่านฮาร์ดแวร์และ Web Interface พร้อมระบบบันทึกและเล่นซ้ำการเคลื่อนที่ (Record & Replay)**

เปรียบเทียบงานวิจัยที่เกี่ยวข้อง

บทที่ 3

ขั้นตอนและวิธีการดำเนินงาน

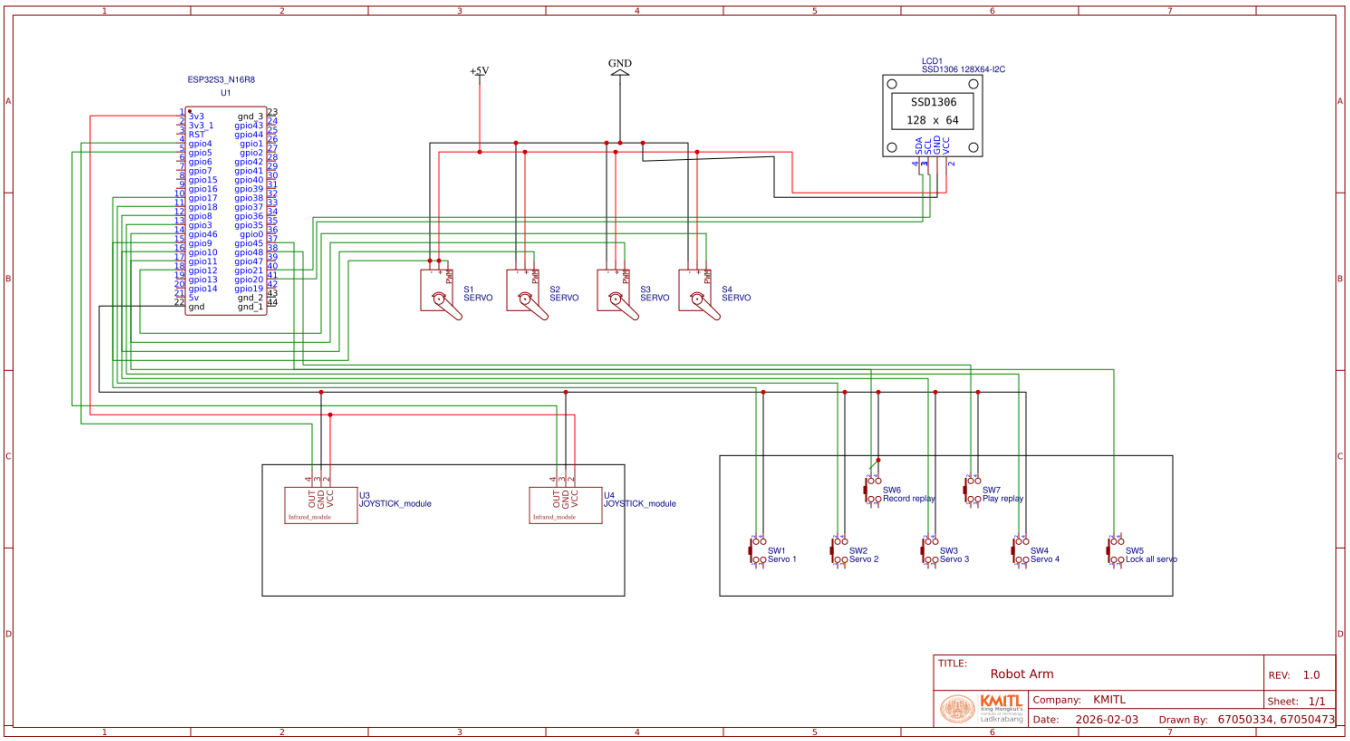
3.1 รายการอุปกรณ์

อุปกรณ์	จำนวน
ESP32-S3	1
Module Joystick	2
Bread Board	2
สายไฟ	50
ปุ่มเล็ก	5
ปุ่มใหญ่	2
จอ OLED	1
Servo MG90S	4
ชิ้นส่วนแขนหุ่นยนต์	13
น็อต	50

3.2การต่อวงจร

อุปกรณ์	Pin	หน้าที่
Servo Motor 1 (MG90S)	15	มอเตอร์แกนที่ 1
Servo Motor 2 (MG90S)	16	มอเตอร์แกนที่ 2
Servo Motor 3 (MG90S)	17	มอเตอร์แกนที่ 3
Servo Motor 4 (MG90S)	18	มอเตอร์แกนที่ 4 ตัวหนีบ
Joystick 1 แกน X	4	ควบคุม Servo 2/3
Joystick 2 แกน X	5	ควบคุม Servo 1
OLED Display (SCL)	40	สัญญาณ Clock ของหน้าจอ
OLED Display (SDA)	41	สัญญาณ Data ของหน้าจอ
ปุ่มเล็ก 1	Pin 10 (Input Pull-up)	Lock/Unlock Servo 1
ปุ่มเล็ก 2	Pin 11 (Input Pull-up)	เลือกควบคุม Servo 2
ปุ่มเล็ก 3	Pin 12 (Input Pull-up)	เลือกควบคุม Servo 3
ปุ่มเล็ก 4	Pin 13 (Input Pull-up)	เปิด/ปิด ตัวหนีบ
ปุ่มเล็ก 5	Pin 14 (Input Pull-up)	Global Lock หยุดทุกแกน
ปุ่มใหญ่ 1	Pin 48 (Input Pull-up)	เริ่ม/หยุด บันทึกท่าทาง
ปุ่มใหญ่ 2	Pin 47 (Input Pull-up)	เล่นซ้ำท่าทางที่บันทึกไว้
Boot button	Pin 0 (Input Pull-up)	เปลี่ยน Mode (Physical/WiFi/Off)

schematics


ESP32S3_N16R8
U1
+5V
GND
LCD1
SSD1306 128X64-I2C
SSD1306
128 x 64
SDA
SCL
GND
VCC
S1
SERVO
S2
SERVO
S3
SERVO
S4
SERVO
U3
JOYSTICK_module
U4
JOYSTICK_module
OUT
GND
VCC
Infrared_module
SW1
Servo 1
SW2
Servo 2
SW3
Servo 3
SW4
Servo 4
SW5
Lock all servo
SW6
Record replay
SW7
Play replay
TITLE:
Robot Arm
REV: 1.0
Company: KMITL
Sheet: 1/1
Date: 2026-02-03
Drawn By: 67050334, 67050473

3.3หลักการการทำงาน

1. **การรับสัญญาณอินพุตจาก Joystick**

Module Joystick ทำงานพื้นฐานด้วยตัวต้านทานปรับค่าได้ เมื่อผู้ใช้งานโยกก้านจอยสติ๊ก แรงดันไฟฟ้า ที่ส่งออกมาจะเกิดการเปลี่ยนแปลงในรูปแบบของ สัญญาณ Analog (ระหว่าง 0 ถึง 3.3V) ไมโครคอนโทรลเลอร์ (ESP32S3) ไม่สามารถประมวลผลค่า Analog ได้โดยตรง จึงต้องส่งสัญญาณนี้ผ่านโมดูล ADC (Analog-to-Digital Converter) เพื่อแปลงขนาดของแรงดันไฟฟ้าให้เป็น ข้อมูลดิจิทัล (Digital Value) โดยในระบบนี้ใช้ความละเอียดแบบ 12-bit ค่าที่อ่านได้จึงอยู่ในช่วง 0 ถึง 4095

2. **การประมวลผลและการตั้งค่าจุดศูนย์กลาง**

2.1 เมื่อเปิดระบบจะทำการอ่านค่าตำแหน่งเริ่มต้นของจอยสติ๊กเพื่อบันทึกเป็น "จุดศูนย์กลาง" (Center Point: cx1, cx2) หลังจากนั้น เมื่อมีการใช้งานจริง ระบบจะคำนวณหาระยะการขยับ (Delta) ด้วยสมการ:

$$\text{Delta} = \text{Current_ADC_Value} - \text{Center_Point}$$

2.2 ระบบได้มีการออกแบบกลไก Deadzone ในโค้ดกำหนดค่าไว้ที่ 800 เพื่อแก้ปัญหาสัญญาณรบกวน หากค่าสัมบูรณ์ของ Delta มีค่าน้อยกว่า Deadzone (Delta < 800) ระบบจะถือว่าไม่มีการขยับจอยสติ๊ก เพื่อป้องกันไม่ให้แขนหุ่นยนต์สั่นหรือขยับเองเมื่อผู้ใช้ปล่อยมือ

3. **คณิตศาสตร์ของการสร้างสัญญาณ PWM**

Servo ในโครงงานนี้ใช้ความถี่ในการทำงานที่ 50Hz ดังนั้นคาบเวลาของคลื่นสัญญาณ 1 รอบ สามารถคำนวณได้ดังนี้

$$T = 1 / f$$

$$T = 1 / 50 = 0.02 \text{ Seconds} = 20\text{ms}$$

ในการสั่งงาน ไมโครคอนโทรลเลอร์ใช้ฟังก์ชันการสร้างคลื่น PWM ด้วยความละเอียดระดับ 16-bit (duty_u16) ซึ่งหมายความว่าใน 1 คาบเวลา (20 ms) จะถูกแบ่งความละเอียดของการจ่ายไฟออกเป็น 0 ถึง 65535

การหาความกว้างของพัลส์ ที่ต้องการส่งไปยังเซอร์โว เพื่อคำนวณจุดหยุดนิ่งสามารถคำนวณได้จากสมการ:

$$\text{Duty_Value} = (\text{t-pulse} / T) \times 65535$$

ตัวอย่างการคำนวณจุดหยุดนิ่ง (Stop Position)

มาตรฐานของเซอร์โวมอเตอร์ จะหยุดนิ่งเมื่อได้รับความกว้างพัลส์ t-pulse = 1.5ms เมื่อนำมาแทนค่าในสมการจะได้:

$$\text{Duty_Value} = (1.5 / 20) \times 65535 \approx 4915$$

นี่คือที่มาของค่าคงที่ 4915 ในระบบที่ใช้ในการหยุดการเคลื่อนที่ของข้อต่อหุ่นยนต์

4. การแมปปิ้งคำสั่งขับมอเตอร์

เซอร์โวมอเตอร์ที่ใช้เป็นชนิดหมุนต่อเนื่อง (Continuous Rotation) สัญญาณ PWM จึงไม่ได้กำหนด "องศา" แต่เป็นการกำหนด "ทิศทางและความเร็ว" โครงงานนี้ได้ออกแบบค่า Duty ไว้ 5 ระดับ เพื่อควบคุมพลศาสตร์ของหุ่นยนต์ ดังตารางต่อไปนี้:

สถานะการทำงาน	ความกว้างพัลส์ โดยประมาณ (t-pulse)	ค่า Duty 16-bit	ผลลัพธ์ทางกลศาสตร์
FAST_CW	≈1.0 ms	3370	มอเตอร์หมุนตามเข็มนาฬิกา (เร็ว) / คลายก้ามปู
SLOW_CW	≈1.28 ms	4200	มอเตอร์หมุนตามเข็มนาฬิกา (ช้า)
STOP_DUTY	1.5 ms	4915	มอเตอร์หยุดนิ่ง
SLOW_CCW	≈1.7 ms	5600	มอเตอร์หมุนทวนเข็มนาฬิกา (ช้า)
FAST_CCW	≈2.0 ms	6700	มอเตอร์หมุนทวนเข็มนาฬิกา (เร็ว) / หนีบก้ามปู

5. กลไกการบันทึกข้อมูลและการเล่นซ้ำ

5.1 กระบวนการบันทึกข้อมูล (Data Logging / Record Mode)

การบันทึกข้อมูลไม่ได้ใช้วิธีการบันทึกค่าตลอดเวลา (Continuous Logging) ซึ่งจะทำให้สิ้นเปลืองพื้นที่หน่วยความจำและทำให้ระบบทำงานช้าลง แต่ซอฟต์แวร์ได้ถูกออกแบบมาให้ใช้หลักการ Event-Driven Logging การบันทึกเมื่อมีการเปลี่ยนแปลงสถานะ

ขั้นตอนการทำงานมีดังนี้:

1. **เริ่มต้นบันทึก:** เมื่อผู้ใช้กดปุ่ม Record ระบบจะล้างข้อมูลเก่าในไฟล์ replay.txt ทิ้ง (เขียนทับด้วยไฟล์ว่าง) และบันทึกเวลาเริ่มต้นระบบไว้ในตัวแปร start_time โดยใช้ฟังก์ชันอ่านค่ามิลลิวินาทีของฮาร์ดแวร์ time.ticks_ms()
2. **การตรวจจับการเปลี่ยนแปลง:** ในลูปการทำงานปกติ เมื่อมีการส่งคำสั่งขับมอเตอร์ ฟังก์ชัน write_log() จะถูกเรียกใช้งาน โปรแกรมจะนำค่า Duty_Value ล่าสุดไปเปรียบเทียบกับค่าในตัวแปร last_duty หากค่าเท่าเดิม โปรแกรมจะข้ามการทำงานไป เพื่อไม่ให้เกิดการเขียนข้อมูลซ้ำซ้อน
3. **การจัดเก็บข้อมูล:** หากมีการเปลี่ยนแปลงความเร็วหรือทิศทาง โปรแกรมจะคำนวณเวลาที่ผ่านไป จากจุดเริ่มต้นด้วยฟังก์ชัน time.ticks_diff() และนำข้อมูล 3 ส่วนประกอบเข้าด้วยกันในรูปแบบไฟล์ CSV ได้แก่:
    - Timestamp (t): เวลาที่เกิดคำสั่ง (หน่วยมิลลิวินาที)
    - Pin: หมายเลขขาพินของมอเตอร์ที่ถูกสั่งงาน (เช่น 15, 16, 17, 18)
    - Duty: ค่าสัญญาณ PWM ที่ส่งออกไป
4. **การเขียนลงไฟล์:** ข้อมูลจะถูกเขียนต่อท้าย ลงในไฟล์ replay.txt เช่น 1500,15,5600 (ความหมาย: ที่เวลา 1.5 วินาที สั่งให้พิน 15 จ่ายค่าพัลส์ 5600) จากนั้นระบบจะอัปเดตค่า last_duty เพื่อเตรียมรับการเปลี่ยนแปลงครั้งต่อไป

5.2 กระบวนการเล่นซ้ำ (Replay Mechanism)

เมื่อผู้ใช้กดปุ่ม Replay ระบบจะทำหน้าที่เป็นเสมือนเครื่องเล่นแผ่นเสียงที่อ่านข้อมูลจากไฟล์ replay.txt แล้วแปลงกลับเป็นสัญญาณไฟฟ้าทางกายภาพ โดยมีความท้าทายหลักคือ การซิงโครไนซ์เวลาเพื่อให้หุ่นยนต์ขยับด้วยความเร็วและจังหวะที่ตรงกับต้นฉบับทุกประการ

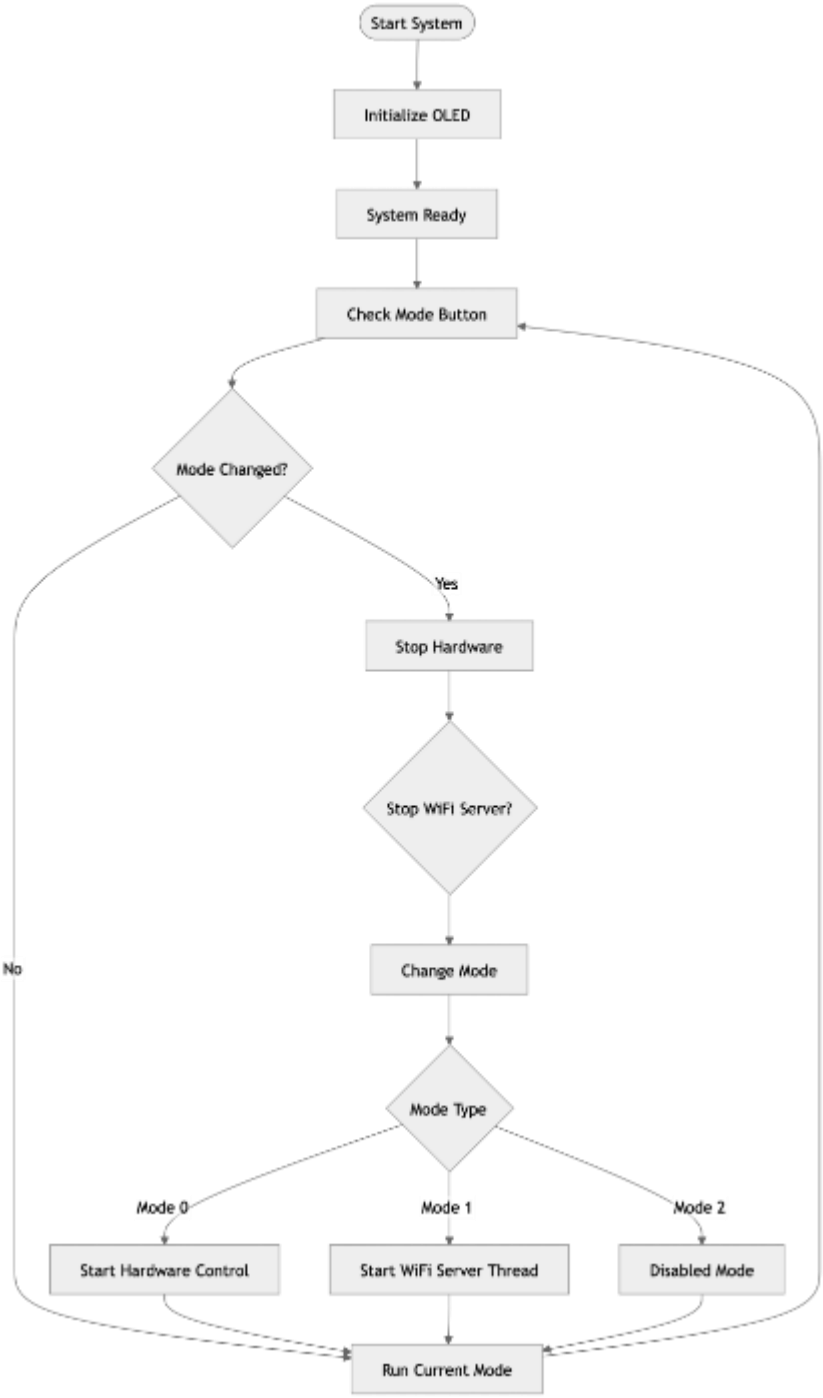
ขั้นตอนการทำงานมีดังนี้:

1. **การเตรียมความพร้อม:** ระบบจะเรียกใช้ฟังก์ชัน stop_all() เพื่อหยุดการเคลื่อนไหวของมอเตอร์ทุกตัวและรอ 200 มิลลิวินาที เพื่อป้องกันการกระชากของกระแสไฟฟ้า
2. **การตั้งค่าเวลาอ้างอิง:** โปรแกรมจะอ่านเวลาเริ่มต้นของการ Replay เก็บไว้ในตัวแปร replay_start
3. **การอ่านและแยกข้อมูล:** ระบบจะอ่านไฟล์ replay.txt ทีละ และใช้คำสั่ง split(",") เพื่อแยกข้อมูล Timestamp, Pin และ Duty ออกจากกัน
4. **การหน่วงเวลาให้ตรงจังหวะ:** โปรแกรมจะเข้าสู่ลูปเงื่อนไข

   t_current - t_replay_start < t_recorded

   ลูปนี้ จะทำงานวนซ้ำไปเรื่อยๆ จนกว่าเวลาจริงของระบบจะเดินมาถึงเวลาที่ระบุไว้ในไฟล์ข้อมูลแล้วไปที่บรรทัดต่อไป

Flowchart


Start System
Initialize OLED
System Ready
Check Mode Button
Mode Changed?
Yes
No
Stop Hardware
Stop WiFi Server?
Change Mode
Mode Type
Mode 0
Mode 1
Mode 2
Start Hardware Control
Start WiFi Server Thread
Disabled Mode
Run Current Mode

3.5 Source code

project_iot_y2.py: โค้ดนี้คือ โปรแกรมหลัก ที่ทำหน้าที่เป็น "ตัวสลับโหมดการทำงาน" ของหุ่นยนต์ พร้อมกับแสดงผลสถานะออกทางหน้าจอ OLED

Mode 0 (Hardware): สั่งให้หุ่นยนต์อ่านค่าและรับคำสั่งจาก "จอยสติ๊ก"

Mode 1 (WiFi): สั่งเปิดเว็บเซิร์ฟเวอร์แบบเบื้องหลัง (Background Thread) เพื่อรอรับคำสั่งจาก "มือถือ/หน้าเว็บ"

Mode 2 (Disabled): โหมดหยุดพัก (Standby) ตัดการทำงานทุกอย่างเพื่อความปลอดภัย

project_iot_y2.py

```
1. from machine import Pin, I2C
2. from time import sleep
3. from ssd1306 import SSD1306_I2C
4. import mode
5. import Project_IoT_Y2_V2
6. import Project_IoT_Y2_V2_wifi
7. import test
8. import _thread
9.
10. # ================= OLED INIT =================
11. def init_oled():
12.     global i2c, oled
13.     try:
14.         i2c = I2C(1, scl=Pin(40), sda=Pin(41), freq=100000)
15.         oled = SSD1306_I2C(128, 64, i2c, addr=0x3c)
16.         print("OLED Ready")
17.     except Exception as e:
18.         print("OLED Error:", e)
19.         oled = None
20.
21. # ================= WIFI SERVER CONTROL =================
22. server_running = False
23. server_thread = None
24.
25. def run_wifi_server():
26.     """รัน WiFi server ในแยก"""
27.     global server_running
28.     try:
29.         Project_IoT_Y2_V2_wifi.start_server()
30.     except Exception as e:
31.         print("WiFi Server Error:", e)
32.     finally:
33.         server_running = False
34.         print("WiFi Server Stopped")
35.
36. # ================= STARTUP =================
37. oled = None
38. i2c = None
39. init_oled()
40. current_mode = -1
41. print("=== SYSTEM READY ===")
42.
43. # ================= MAIN LOOP =================
44. while True:
45.     new_m = mode.update_mode()
```

project_iot_y2.py (main)

```
    # -------- Mode Change --------
    if new_m != current_mode:
        current_mode = new_m
        print("Mode:", current_mode)

        # ========== หยุดทุกอย่างก่อน ==========
        try:
            Project_IoT_Y2_V2.stop_all()
        except Exception as e:
            print("Error stopping hardware:", e)

        # หยุด WiFi server ถ้ากำลังทำงาน
        if server_running:
            try:
                Project_IoT_Y2_V2_wifi.stop_all()
                server_running = False
                sleep(0.5)  # รอให้ server หยุดจริง ๆ
                print("WiFi Server stopped")
            except Exception as e:
                print("Error stopping WiFi server:", e)
                server_running = False

        # ========== MODE 0: HARDWARE MODE ==========
        if current_mode == 0:
            try:
                Project_IoT_Y2_V2.start_all()
                if oled:
                    test.display_mode0(oled)
                print("HARDWARE MODE started")
            except Exception as e:
                print("Error starting hardware mode:", e)

        # ========== MODE 1: WIFI MODE ==========
        elif current_mode == 1:
            try:
                if oled:
                    test.display_mode1(oled)
                print("Entering WiFi Mode...")
                sleep(0.5)

                # รัน server ใน thread แยก
                server_running = True
                server_thread = _thread.start_new_thread(run_wifi_server, ())
                print("WiFi Server started in background thread")
            except Exception as e:
                print("Error starting WiFi mode:", e)
                server_running = False

        # ========== MODE 2: DISABLED ==========
        elif current_mode == 2:
            try:
                if oled:
                    test.display_disabled(oled)
                print("DISABLED MODE")
            except Exception as e:
                print("Error in disabled mode:", e)

    # -------- RUN LOOP --------
    try:
        if current_mode == 0:
            # HARDWARE MODE
            Project_IoT_Y2_V2.update()
```

```
                                project_iot_y2.py   (main)          == 1:
            # WIFI MODE - server running in background
            sleep(0.1)
        elif current_mode == 2:
            # DISABLED MODE
            sleep(0.1)
    except Exception as e:
        print("Error in main loop:", e)

    sleep(0.02)

```

mode.py: ระบบสลับโหมดการทำงานด้วยปุ่มกด โค้ดส่วนนี้ทำหน้าที่เป็นระบบสลับโหมดการทำงานผ่านปุ่มกดเดียว โดยอาศัยการตรวจจับระยะเวลาการกดปุ่ม ที่ขาสัญญาณ Pin 0 (ปุ่ม BOOT ของบอร์ด) เพื่อเปลี่ยนสถานะการทำงานของหุ่นยนต์

โปรแกรมจะทำงานโดยแบ่งเงื่อนไขตามระยะเวลาการกด ดังนี้:

- กดค้าง (นานกว่า 600 มิลลิวินาที): ใช้สำหรับ สลับโหมดการควบคุมหลัก ไปมา ระหว่างโหมดควบคุมผ่านมือถือ (Mode 0: MOBILE) และโหมดควบคุมผ่านจอยสติ๊ก (Mode 1: HARDWARE)
- กดสั้น (น้อยกว่า 600 มิลลิวินาที): ใช้สำหรับ หยุดการทำงานฉุกเฉินหรือสแตนด์บาย โดยจะบังคับให้ระบบเข้าสู่โหมดพัก (Mode 2: OFF) ทันทีเพื่อความปลอดภัย

mode.py

```
 1. from machine import Pin
 2. import time
 3.
 4. boot = Pin(0, Pin.IN, Pin.PULL_UP)
 5.
 6. mode = 0  # 0=MOBILE, 1=HARDWARE, 2=OFF
 7.
 8. press_time = None
 9. LONG_PRESS_MS = 600
10.
11.
12. def update_mode():
13.     global mode, press_time
14.
15.     if boot.value() == 0:  # กดอยู่
16.         if press_time is None:
17.             press_time = time.ticks_ms()
18.
19.     else:  # ปล่อยแล้ว
20.         if press_time is not None:
21.             duration = time.ticks_diff(time.ticks_ms(), press_time)
22.             press_time = None
23.
24.             # ----- LONG PRESS -----
25.             if duration > LONG_PRESS_MS:
26.                 if mode == 0:
27.                     mode = 1
28.                 else:
29.                     mode = 0
30.
31.             # ----- SHORT PRESS -----
32.             else:
33.                 mode = 2
34.
35.     return mode
36.
```

Project_IoT_Y2_V2.py: ระบบควบคุมแขนหุ่นยนต์ด้วย Module Joystick ซอฟต์แวร์ส่วนนี้ทำหน้าที่รับสัญญาณอินพุตจากผู้ใช้งานผ่านโมดูลจอยสติ๊กและปุ่มกดทางกายภาพ เพื่อแปลงเป็นคำสั่งขับเคลื่อนข้อต่อต่างๆ ของหุ่นยนต์ โดยมีฟังก์ชันการทำงานหลัก 5 ส่วน ดังนี้:

1. การควบคุมการเคลื่อนที่: อ่านค่าแอนะล็อก (ADC) จากจอยสติ๊ก 2 ตัว นำมาหักล้างกับค่า Deadzone เพื่อลดสัญญาณรบกวน แล้วแปลงเป็นความกว้างคลื่น PWM ส่งไปสั่งงาน Servo Motor ให้หมุนซ้าย/ขวา ช้าหรือเร็วตามระยะการโยก
2. ระบบสลับแกนควบคุม: เนื่องจากมีจอยสติ๊กจำกัด จึงออกแบบให้มีปุ่มกด (Pin 11, 12) สำหรับสลับการควบคุม ระหว่างข้อต่อที่ 2 (Pin 16) และข้อต่อที่ 3 (Pin 17) ทำให้สามารถควบคุมแขนได้ครบทุกแกน
3. ระบบความปลอดภัยและการล็อคแกน: มีปุ่ม Global Lock (Pin 14) สำหรับทำหน้าที่เป็นปุ่มหยุดฉุกเฉิน และปุ่มล็อคแกนฐาน (Pin 10) เพื่อป้องกันการหมุนโดยไม่ตั้งใจ
4. การควบคุมชุดหนีบจับ: มีปุ่มกด (Pin 13) สำหรับสั่งงานเปิดและปิดก้ามปู (Pin 18) โดยมีการหน่วงเวลา 350 มิลลิวินาทีในแต่ละจังหวะเพื่อให้มอเตอร์ทำงานได้สมบูรณ์ก่อนตัดกระแสไฟ (STOP_DUTY)
5. ระบบ Record & Replay: มีปุ่มกดเฉพาะสำหรับ เริ่ม/หยุด การบันทึก (Pin 48) และปุ่มเล่นซ้ำ (Pin 47) โดยอาศัยการจดบันทึกการเปลี่ยนแปลงของสัญญาณลงในไฟล์ replay.txt ตามที่ได้ออกแบบไว้

Project_IoT_Y2_V2.py

```
1. from machine import Pin, PWM, ADC
2. import time
3. import os
4.
5. # ================= CONFIG =================
6. PIN_SERVO_15 = 15
7. PINS_OTHERS = [16, 17, 18]
8.
9. STOP_DUTY = 4915
10. FAST_CW, FAST_CCW = 4200, 5600
11. SLOW_CW, SLOW_CCW = 3370, 6700
12. DEADZONE = 800
13.
14. GRIP = 6700
15. RELEASE = 3370
16.
17. RECORD_FILE = "replay.txt"
18.
19. # ================= HARDWARE =================
20. servo_15 = PWM(Pin(PIN_SERVO_15), freq=50)
21. servos = {p: PWM(Pin(p), freq=50) for p in PINS_OTHERS}
22.
23. joy1_x = ADC(Pin(4)); joy1_x.atten(ADC.ATTN_11DB)
24. joy2_x = ADC(Pin(5)); joy2_x.atten(ADC.ATTN_11DB)
25.
26. buttons = [Pin(p, Pin.IN, Pin.PULL_UP) for p in [10,11,12,13,14]]
27.
28. rec_btn = Pin(48, Pin.IN, Pin.PULL_UP)
```

Project_IoT_Y2_V2.py

```
replay_btn = Pin(47, Pin.IN, Pin.PULL_UP)

# ================= STATE =================
selected_idx = 0
servo15_locked = False
global_lock = False
gripper_closed = False

is_recording = False
start_time = 0
last_duty = {}

cx1, cx2 = joy1_x.read(), joy2_x.read()

# ================= BASIC FUNCTIONS =================

def stop_all():
    servo_15.duty_u16(STOP_DUTY)
    for p in servos:
        servos[p].duty_u16(STOP_DUTY)

def get_duty(pin, delta):

    if abs(delta) < DEADZONE:
        return STOP_DUTY

    if pin in [15,17]:
        return FAST_CW if delta > 0 else FAST_CCW

    return SLOW_CW if delta > 0 else SLOW_CCW


# ================= RECORD =================

def write_log(pin, duty):

    if not is_recording:
        return

    ts = time.ticks_diff(time.ticks_ms(), start_time)

    if last_duty.get(pin) == duty:
        return

    with open(RECORD_FILE, "a") as f:
        f.write("{},{},{}\n".format(ts, pin, duty))

    last_duty[pin] = duty


# ================= REPLAY =================

def play_replay():

    if RECORD_FILE not in os.listdir():
        print("No replay file")
        return

    stop_all()
    time.sleep_ms(200)

    with open(RECORD_FILE, "r") as f:

        replay_start = time.ticks_ms()
```

Project_IoT_Y2_V2.py

```python

        for line in f:

            parts = line.strip().split(",")

            if len(parts) != 3:
                continue

            t,pin,duty = map(int,parts)

            while time.ticks_diff(time.ticks_ms(), replay_start) < t:
                pass

            if pin == 15:
                servo_15.duty_u16(duty)

            elif pin in servos:
                servos[pin].duty_u16(duty)

    stop_all()
    print("Replay Done")


# ================= GRIPPER =================

def toggle_gripper():
    global gripper_closed

    if gripper_closed:

        duty = RELEASE
        servos[18].duty_u16(duty)
        write_log(18, duty)

        time.sleep_ms(350)

        servos[18].duty_u16(STOP_DUTY)
        write_log(18, STOP_DUTY)

        gripper_closed = False
        print("Gripper Open")

    else:

        duty = GRIP
        servos[18].duty_u16(duty)
        write_log(18, duty)

        time.sleep_ms(350)

        servos[18].duty_u16(STOP_DUTY)
        write_log(18, STOP_DUTY)

        gripper_closed = True
        print("Gripper Close")


# ================= START =================

def start_all():
    print("[HARDWARE MODE]")
    stop_all()


```

Project_IoT_Y2_V2.py

```
# ================= UPDATE LOOP =================

def update():

    global selected_idx, servo15_locked, global_lock
    global is_recording, start_time

    # ===== GLOBAL LOCK =====
    if buttons[4].value() == 0:
        global_lock = not global_lock
        stop_all()
        time.sleep_ms(300)

    if global_lock:
        stop_all()
        return

    # ===== SELECT SERVO =====
    for i in [1,2]:
        if buttons[i].value() == 0:
            selected_idx = i - 1
            time.sleep_ms(250)

    # ===== JOYSTICK CONTROL S16/S17 =====
    current_pin = PINS_OTHERS[selected_idx]

    dx1 = joy1_x.read() - cx1

    duty = get_duty(current_pin, dx1)

    servos[current_pin].duty_u16(duty)
    write_log(current_pin, duty)

    # ===== LOCK SERVO 15 =====
    if buttons[0].value() == 0:
        servo15_locked = not servo15_locked
        time.sleep_ms(300)

    # ===== GRIPPER =====
    if buttons[3].value() == 0:
        toggle_gripper()
        time.sleep_ms(300)

    # ===== RECORD BUTTON =====
    if rec_btn.value() == 0:

        is_recording = not is_recording

        if is_recording:

            with open(RECORD_FILE, "w") as f:
                f.write("")

            start_time = time.ticks_ms()
            last_duty.clear()

            print("Recording Start")

        else:
            print("Recording Stop")

        time.sleep_ms(400)

    # ===== REPLAY BUTTON =====
```

Project_IoT_Y2_V2.py

```
    if replay_btn.value() == 0:
        play_replay()
        time.sleep_ms(400)

    # ===== JOYSTICK CONTROL SERVO15 =====
    if not servo15_locked:

        dx2 = joy2_x.read() - cx2

        duty = get_duty(15, dx2)

        servo_15.duty_u16(duty)
        write_log(15, duty)

    else:
        servo_15.duty_u16(STOP_DUTY)

```

Project_IoT_Y2_V2_wifi.py: ระบบควบคุมแขนหุ่นยนต์ผ่าน WiFi Web Server

การทำงานเป็น 3 ส่วนหลัก ดังนี้:

1. การสร้างเครือข่าย (WiFi AP Setup): ระบบจะกระจายสัญญาณ WiFi ในชื่อ "ESP32-Robot" เพื่อให้อุปกรณ์ภายนอกเชื่อมต่อเข้ามายังวงแลนเดียวกัน
2. การให้บริการหน้าเว็บเพจ: เมื่อผู้ใช้งานเข้าถึงหมายเลข IP ของบอร์ด (เช่น 192.168.4.1) ระบบจะทำหน้าที่เป็นโฮสต์ ส่งไฟล์ HTML, CSS, และ JavaScript ที่เก็บไว้ในหน่วยความจำของบอร์ดไปแสดงผลเป็น "หน้าจอควบคุมเสมือน (Virtual Controller UI)" บนเบราว์เซอร์
3. การรับคำสั่งควบคุม (API Handler): เมื่อผู้ใช้กดปุ่มบนหน้าเว็บ เบราว์เซอร์จะส่ง HTTP Request (เช่น /s15_cw, /stop, /record) กลับมายังบอร์ด โปรแกรมจะทำหน้าที่แปลความหมาย (Parsing) ของ URL (Path) เหล่านั้นให้เป็นคำสั่งทางฮาร์ดแวร์เพื่อขับเคลื่อนมอเตอร์ บันทึกการทำงาน หรือเล่นซ้ำแบบเรียลไทม์

Project_IoT_Y2_V2_wifi.py

```
from machine import Pin, PWM
import time
import network
import socket
import os

# =========================
# WIFI AP SETUP
# =========================
ap = network.WLAN(network.AP_IF)
ap.active(True)
ap.config(essid="ESP32-Robot", password="12345678", authmode=3)

while not ap.active():
    pass

print("AP IP:", ap.ifconfig()[0])

# =========================
# SERVO CONFIG (Continuous)
# =========================
STOP_DUTY = 4915
FAST_CW   = 4200
FAST_CCW  = 5600
SLOW_CW   = 3370
SLOW_CCW  = 6700

RECORD_FILE = "replay.txt"

servos = {
    15: PWM(Pin(15), freq=50),
    16: PWM(Pin(16), freq=50),
    17: PWM(Pin(17), freq=50),
    18: PWM(Pin(18), freq=50)
```

Project_IoT_Y2_V2_wifi.py

```
}

last_duty = {15: STOP_DUTY, 16: STOP_DUTY, 17: STOP_DUTY, 18: STOP_DUTY}
is_recording = False
start_time = 0

# =========================
# SERVO CONTROL
# =========================
def set_servo(pin, duty):
    global last_duty
    duty = max(3000, min(7000, duty))

    if pin in servos:
        servos[pin].duty_u16(duty)

        if is_recording and duty != last_duty[pin]:
            write_log(pin, duty)
            last_duty[pin] = duty

def stop_all():
    for pin in servos:
        servos[pin].duty_u16(STOP_DUTY)
    print("All Servos Stopped")

# =========================
# RECORD / REPLAY
# =========================
def write_log(pin, duty):
    ts = time.ticks_diff(time.ticks_ms(), start_time)
    with open(RECORD_FILE, "a") as f:
        f.write("{},{},{}\n".format(ts, pin, duty))

def play_replay():
    global is_recording
    if is_recording:
        return

    try:
        stop_all()
        time.sleep(0.5)

        with open(RECORD_FILE, "r") as f:
            replay_start = time.ticks_ms()

            for line in f:
                parts = line.strip().split(",")
                if len(parts) != 3:
                    continue

                t = int(parts[0])
                pin = int(parts[1])
                duty = int(parts[2])

                while time.ticks_diff(time.ticks_ms(), replay_start) < t:
                    time.sleep_ms(1)

                if pin in servos:
                    servos[pin].duty_u16(duty)

        stop_all()
        print("Replay Done")

    except Exception as e:
```

Project_IoT_Y2_V2_wifi.py

```
        print("Replay Error:", e)
        stop_all()

# ========================
# API HANDLER
# ========================
def handle_api(path):
    global is_recording, start_time
    print("PATH:", path)

    try:
        # ===== SERVO COMMANDS =====
        if path == "/s15_cw":   set_servo(15, SLOW_CW)
        elif path == "/s15_ccw": set_servo(15, SLOW_CCW)

        elif path == "/s16_cw":   set_servo(16, SLOW_CW)
        elif path == "/s16_ccw":  set_servo(16, SLOW_CCW)

        elif path == "/s17_cw":   set_servo(17, FAST_CW)
        elif path == "/s17_ccw":  set_servo(17, FAST_CCW)

        elif path == "/s18_cw":   set_servo(18, SLOW_CW)
        elif path == "/s18_ccw":  set_servo(18, SLOW_CCW)

        # ===== SYSTEM =====
        elif path == "/stop":
            stop_all()

        elif path == "/record":
            is_recording = not is_recording
            if is_recording:
                with open(RECORD_FILE, "w") as f:
                    f.write("")
                start_time = time.ticks_ms()
                print("Recording Start")
            else:
                print("Recording Stop")

        elif path == "/replay":
            play_replay()

    except Exception as e:
        print("API Error:", e)

# ========================
# STATIC FILE SENDER
# ========================
def send_file(cl, filename, content_type):
    try:
        with open(filename, "rb") as f:
            cl.send("HTTP/1.1 200 OK\r\nContent-Type: {}\r\n\r\n".format(content_type))
            while True:
                chunk = f.read(1024)
                if not chunk:
                    break
                cl.send(chunk)
    except:
        cl.send("HTTP/1.1 404 Not Found\r\n\r\nFile Not Found")

# ========================
# WEB SERVER
# ========================
def start_server():
    s = socket.socket()
```

Project_IoT_Y2_V2_wifi.py

```
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(('0.0.0.0', 80))
    s.listen(1)

    print("Server Ready")

    while True:
        cl, addr = s.accept()
        try:
            req = cl.recv(1024).decode()
            if not req:
                cl.close()
                continue

            path = req.split(" ")[1]
            print("PATH:", path)

            # ===== ROOT =====
            if path == "/" or path == "/index.html":
                send_file(cl, "dist/index.html", "text/html")

            # ===== ASSETS =====
            elif path.startswith("/assets/"):
                filename = "dist" + path
                if filename.endswith(".js"):
                    send_file(cl, filename, "application/javascript")
                elif filename.endswith(".css"):
                    send_file(cl, filename, "text/css")
                elif filename.endswith(".svg"):
                    send_file(cl, filename, "image/svg+xml")
                else:
                    send_file(cl, filename, "application/octet-stream")

            # ===== API =====
            elif path.startswith("/s") or path in ["/stop","/record","/replay"]:
                handle_api(path)
                cl.send("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\nOK")

            else:
                cl.send("HTTP/1.1 404 Not Found\r\n\r\n")

        except Exception as e:
            print("Server Error:", e)

        cl.close()

```

test.py: ระบบแสดงผลสถานะการทำงานผ่านจอ OLED

ซอฟต์แวร์ส่วนนี้ทำหน้าที่ควบคุมหน้าจอแสดงผลผ่านโปรโตคอลการสื่อสาร I2C เพื่อเป็นส่วนติดต่อผู้ใช้งาน ให้ทราบถึงสถานะการทำงานปัจจุบันของหุ่นยนต์ โดยแบ่งการแสดงผลตามโหมดต่างๆ ดังนี้:

1. การสร้างกราฟิกรูปภาพ (Bitmap Rendering): โปรแกรมใช้วิธีแปลงรูปภาพไอคอน (เช่น รูปจอยสติ๊ก และ รูปสัญญาณ WiFi) ให้อยู่ในรูปแบบอาร์เรย์ฐานสิบหก ขนาด 32x32 พิกเซล เพื่อให้ไมโครคอนโทรลเลอร์สามารถวาดรูปลงบนหน้าจอได้โดยไม่ต้องพึ่งพาไฟล์รูปภาพภายนอก
2. หน้าจอโหมดฮาร์ดแวร์ (Mode 0 - Physical Control): แสดงข้อความ "PHYSICAL CONTROL" พร้อมรูปไอคอนจอยสติ๊ก เพื่อบอกให้ผู้ใช้ทราบว่าหุ่นยนต์พร้อมรับคำสั่งจากปุ่มกดและจอยสติ๊กบนตัวเครื่อง
3. หน้าจอโหมดไร้สาย (Mode 1 - Mobile Control): แสดงข้อความ "MOBILE CONTROL" พร้อมรูปไอคอน WiFi เพื่อแสดงว่าระบบได้เปิดใช้งานเว็บเซิร์ฟเวอร์และพร้อมรับคำสั่งผ่านสมาร์ทโฟนแล้ว
4. หน้าจอโหมดพัก (Disabled Mode): แสดงข้อความ "SYSTEM DISABLED" พร้อมสัญลักษณ์กากบาท เพื่อเตือนว่าระบบปิดการทำงานเพื่อความปลอดภัย (Standby) และมอเตอร์ทุกตัวถูกตัดกระแสไฟแล้ว

test.py

```
1. from machine import Pin, I2C
2. from time import sleep
3. from ssd1306 import SSD1306_I2C
4. import mode
5.
6.
7. print("=== OLED Display with WiFi & Gamepad Icons ===\n")
8.
9. # Initialize I2C and OLED
10. try:
11.     i2c = I2C(1, scl=Pin(40), sda=Pin(41), freq=100000)
12.     oled = SSD1306_I2C(128, 64, i2c, addr=0x3c)
13.     print("✓ OLED initialized at 0x3c\n")
14. except:
15.     try:
16.         oled = SSD1306_I2C(128, 64, i2c, addr=0x3d)
17.         print("✓ OLED initialized at 0x3d\n")
18.     except Exception as e:
19.         print(f"✗ OLED Error: {e}")
20.
21. # WiFi Icon (32x32 pixels)
22. wifi_icon = [
23.     0x00, 0x00, 0x00, 0x00,
24.     0x00, 0x00, 0x00, 0x00,
25.     0x00, 0x00, 0x00, 0x00,
26.     0x00, 0x00, 0x00, 0x00,
27.     0x00, 0x00, 0x00, 0x00,
28.     0x00, 0x01, 0x80, 0x00,
29.     0x00, 0x1f, 0xfc, 0x00,
30.     0x00, 0xff, 0xff, 0x00,
31.     0x01, 0xff, 0xff, 0xc0,
```

test.py

```
    0x03, 0xf0, 0x0f, 0xe0,
    0x0f, 0xc0, 0x03, 0xf0,
    0x0f, 0x0f, 0xf0, 0xf0,
    0x06, 0x3f, 0xfc, 0x60,
    0x00, 0x7f, 0xff, 0x00,
    0x00, 0xfc, 0x3f, 0x80,
    0x00, 0xf0, 0x07, 0x00,
    0x00, 0x47, 0xe2, 0x00,
    0x00, 0x0f, 0xf8, 0x00,
    0x00, 0x1f, 0xf8, 0x00,
    0x00, 0x0e, 0x38, 0x00,
    0x00, 0x00, 0x10, 0x00,
    0x00, 0x01, 0xc0, 0x00,
    0x00, 0x03, 0xe0, 0x00,
    0x00, 0x07, 0xe0, 0x00,
    0x00, 0x07, 0xe0, 0x00,
    0x00, 0x07, 0xe0, 0x00,
    0x00, 0x03, 0xe0, 0x00,
    0x00, 0x01, 0xc0, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00
]

# Gamepad Icon (32x32 pixels)
gamepad_icon = [
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x07, 0x80, 0x0f, 0x80,
    0x1f, 0xe0, 0x1f, 0xc0,
    0x3f, 0xf0, 0x3f, 0xe0,
    0x3f, 0xff, 0xff, 0xf0,
    0x7f, 0xff, 0xff, 0xf0,
    0x7e, 0xff, 0xf9, 0xf8,
    0xfe, 0xff, 0xf8, 0xf8,
    0xfc, 0xff, 0xf8, 0xf8,
    0xf8, 0x3f, 0xe2, 0x3c,
    0xfe, 0xff, 0xe6, 0x3c,
    0xfe, 0xff, 0xe0, 0x7c,
    0xfe, 0xff, 0xf8, 0xfc,
    0xff, 0xff, 0xf8, 0xfc,
    0xff, 0xff, 0xff, 0xfe,
    0xff, 0xff, 0xff, 0xfe,
    0xff, 0xff, 0xff, 0xfe,
    0xff, 0xff, 0xff, 0xfe,
    0xff, 0xff, 0xff, 0xfe,
    0xff, 0xfc, 0xff, 0xfe,
    0xff, 0xc0, 0x1f, 0xfe,
    0xff, 0x80, 0x0f, 0xfe,
    0xff, 0x00, 0x07, 0xfe,
    0xfe, 0x00, 0x03, 0xfe,
    0xfc, 0x00, 0x01, 0xfc,
    0xf8, 0x00, 0x00, 0x7c,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00
]

def draw_bitmap(oled, x, y, bitmap, width=32, height=32):
    """Draw a 32x32 bitmap on OLED display"""
    bytes_per_row = (width + 7) // 8
```

test.py

```

    for row in range(height):
        for byte_idx in range(bytes_per_row):
            byte_val = bitmap[row * bytes_per_row + byte_idx]
            for bit in range(8):
                pixel_x = x + byte_idx * 8 + bit
                pixel_y = y + row

                # Draw if pixel is within display bounds
                if pixel_x < 128 and pixel_y < 64:
                    if byte_val & (1 << (7 - bit)):
                        oled.pixel(pixel_x, pixel_y, 1)

def display_mode0(oled):
    """Mode 0: Physical Control with Gamepad"""
    oled.fill(0)

    # Draw gamepad icon on right side
    draw_bitmap(oled, 65, 8, gamepad_icon)

    # Text on left
    oled.text('PHYSICAL', 0, 8)
    oled.text('CONTROL', 5, 20)

    # Status bar at bottom
    oled.text('Mode: 0', 0, 50)
    oled.text('Active', 70, 50)

    oled.show()

def display_mode1(oled):
    """Mode 1: Mobile Control with WiFi"""
    oled.fill(0)

    # Draw WiFi icon on right side
    draw_bitmap(oled, 75, 10, wifi_icon)

    # Text on left
    oled.text('MOBILE', 10, 8)
    oled.text('CONTROL', 5, 20)

    # Status bar at bottom
    oled.text('Mode: 1', 0, 50)
    oled.text('WiFi', 75, 50)

    oled.show()

def display_disabled(oled):
    """Disabled Mode"""
    oled.fill(0)

    # Center text
    oled.text('SYSTEM', 20, 15)
    oled.text('DISABLED', 15, 30)

    # Draw X mark
    for i in range(12):
        oled.pixel(75 + i, 12 + i, 1)
        oled.pixel(75 + i, 23 - i, 1)

    # Status bar
    oled.text('Mode: X', 0, 50)
    oled.text('OFF', 75, 50)

```

test.py

```
    oled.show()

# Main loop

```

บทที่ 4

ผลการดำเนินงาน

4.1 ภาพรวมของระบบที่พัฒนา

โครงงานนี้ได้พัฒนาระบบแขนกลหุ่นยนต์ที่สามารถควบคุมการเคลื่อนที่ของแขนกลได้ผ่าน 2 รูปแบบ ได้แก่ การควบคุมผ่านอุปกรณ์ฮาร์ดแวร์ (Joystick และปุ่มควบคุม) และการควบคุมผ่าน Web Interface โดยใช้ไมโครคอนโทรลเลอร์ ESP32 เป็นตัวควบคุมหลักของระบบ

ระบบประกอบด้วย Servo Motor จำนวน 4 ตัวสำหรับควบคุมข้อต่อของแขนกล โดยผู้ใช้งานสามารถควบคุมการเคลื่อนที่ของแขนกลเพื่อหยิบจับหรือเคลื่อนย้ายวัตถุได้ตามต้องการ นอกจากนี้ระบบยังมีฟังก์ชันบันทึกและเล่นซ้ำการเคลื่อนที่ของแขนกล (Record and Replay) เพื่อให้สามารถทำงานซ้ำได้โดยอัตโนมัติ

4.2 โครงสร้างของระบบที่พัฒนา

ระบบแขนกลหุ่นยนต์ที่พัฒนาขึ้นประกอบด้วยส่วนประกอบหลักดังนี้

1. ไมโครคอนโทรลเลอร์ **ESP32** สำหรับควบคุมการทำงานของระบบ
2. **Servo Motor จำนวน 4 ตัว** สำหรับควบคุมข้อต่อของแขนกล
3. **Joystick Module** สำหรับควบคุมการเคลื่อนที่ของแขนกลในโหมดฮาร์ดแวร์
4. **Push Button** สำหรับเลือก Servo และสั่งงาน Record / Replay
5. **OLED Display** สำหรับแสดงสถานะของระบบ
6. **Web Interface** สำหรับควบคุมแขนกลผ่านเครือข่าย WiFi

4.3 โหมดการทำงานของระบบ

ระบบที่พัฒนาขึ้นสามารถทำงานได้ทั้งหมด **3 โหมด** ได้แก่

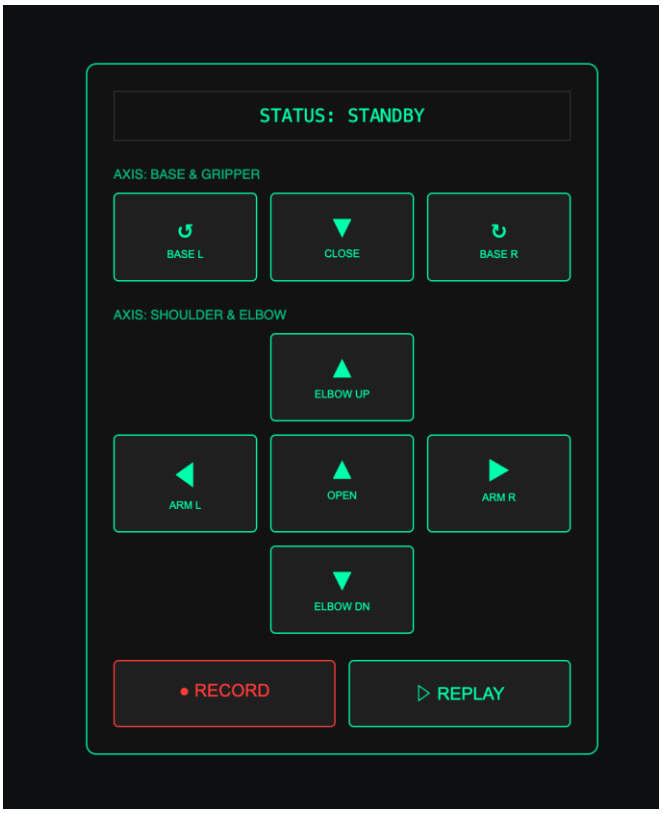
4.3.1 Hardware Control Mode

ในโหมดนี้ผู้ใช้งานสามารถควบคุมแขนกลผ่าน **Joystick และปุ่มกด** โดยผู้ใช้งานสามารถเลือก Servo Motor ที่ต้องการควบคุม และใช้ Joystick ในการควบคุมทิศทางการหมุนของ Servo Motor เพื่อเคลื่อนที่แขนกล

นอกจากนี้ยังมีปุ่มสำหรับ

- บันทึกการเคลื่อนที่ของแขนกล (Record)
- เล่นซ้ำการเคลื่อนที่ที่บันทึกไว้ (Replay)
- หยุดการทำงานของ Servo Motor

4.3.2 WiFi Control Mode



ในโหมดนี้ระบบจะทำการเปิด **WiFi Access Point** บน ESP32 เพื่อให้ผู้ใช้งานสามารถเชื่อมต่อกับระบบผ่านอุปกรณ์ต่าง ๆ เช่น คอมพิวเตอร์หรือสมาร์ตโฟน

เมื่อเชื่อมต่อกับเครือข่าย WiFi ของระบบแล้ว ผู้ใช้งานสามารถเปิด Web Browser และเข้าถึงหน้า

Web Interface เพื่อควบคุมการเคลื่อนที่ของแขนกล เช่น

- หมุน Servo Motor
- หยุดการทำงานของแขนกล
- บันทึกการเคลื่อนที่
- เล่นซ้ำการเคลื่อนที่

4.3.3 Disabled Mode

ในโหมดนี้ระบบจะหยุดการทำงานของ Servo Motor ทั้งหมด เพื่อป้องกันการเคลื่อนที่ของแขนกลโดยไม่ได้ตั้งใจ และช่วยเพิ่มความปลอดภัยในการใช้งาน

4.4 ผลการทดสอบการทำงานของระบบ

จากการทดสอบระบบแขนกลหุ่นยนต์ พบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ โดยมีผลการทดสอบดังนี้

1. ระบบสามารถควบคุม Servo Motor ได้ครบทั้ง 4 ตัว
2. ผู้ใช้งานสามารถควบคุมแขนกลผ่าน Joystick ได้อย่างถูกต้อง
3. ระบบสามารถสร้าง WiFi Access Point และให้ผู้ใช้งานเชื่อมต่อเพื่อควบคุมผ่าน Web Interface ได้
4. ระบบสามารถบันทึกการเคลื่อนที่ของแขนกลและเล่นซ้ำการทำงานได้
5. ระบบสามารถสลับโหมดการทำงานระหว่าง Hardware Mode และ WiFi Mode ได้อย่างถูกต้อง

4.5 สรุปผลการดำเนินงาน

จากการพัฒนาและทดสอบระบบแขนกลหุ่นยนต์ พบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่ตั้งไว้ โดยสามารถควบคุมแขนกลผ่านทั้งอุปกรณ์ฮาร์ดแวร์และ Web Interface ผ่านเครือข่าย WiFi นอกจากนี้ยังมีระบบบันทึกและเล่นซ้ำการเคลื่อนที่ ซึ่งช่วยให้แขนกลสามารถทำงานซ้ำได้โดยอัตโนมัติ

ระบบที่พัฒนาขึ้นสามารถนำไปประยุกต์ใช้ในงานด้านระบบอัตโนมัติ หุ่นยนต์ หรือระบบ Internet of Things (IoT) ได้ในอนาคต

บทที่ 5

สรุปและอภิปรายผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

โครงงานการพัฒนาแขนกลหุ่นยนต์ (Robot Arm) สำหรับการหยิบจับและเคลื่อนย้ายวัตถุ ได้มีการออกแบบและพัฒนาระบบควบคุมโดยใช้ไมโครคอนโทรลเลอร์ **ESP32** เป็นตัวควบคุมหลักของระบบ โดยใช้ **Servo Motor** ในการควบคุมการเคลื่อนที่ของข้อต่อแขนกล เพื่อให้สามารถเคลื่อนที่และหยิบจับวัตถุได้ตามต้องการ ระบบที่พัฒนาขึ้นสามารถควบคุมการทำงานของแขนกลได้ 2 รูปแบบ ได้แก่ การควบคุมผ่านอุปกรณ์ฮาร์ดแวร์โดยใช้ **Joystick และปุ่มควบคุม** และการควบคุมผ่าน **Web Interface ผ่านเครือข่าย WiFi** ซึ่งช่วยเพิ่มความสะดวกในการใช้งานและสามารถควบคุมระบบได้จากอุปกรณ์ต่าง ๆ เช่น คอมพิวเตอร์หรือสมาร์ตโฟน

นอกจากนี้ ระบบยังมีฟังก์ชัน **Record and Replay** สำหรับบันทึกการเคลื่อนที่ของแขนกลและสามารถเล่นซ้ำการทำงานที่บันทึกไว้ได้ ซึ่งช่วยให้แขนกลสามารถทำงานซ้ำได้โดยอัตโนมัติ จากผลการทดสอบพบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ โดยสามารถควบคุม Servo Motor ได้ครบทุกตัว ระบบ WiFi สามารถเชื่อมต่อและควบคุมผ่านหน้าเว็บได้อย่างถูกต้อง และระบบบันทึกและเล่นซ้ำการเคลื่อนที่สามารถทำงานได้อย่างมีประสิทธิภาพ

5.2 อภิปรายผลการดำเนินงาน

จากการพัฒนาและทดสอบระบบแขนกลหุ่นยนต์ พบว่าการใช้ไมโครคอนโทรลเลอร์ **ESP32** สามารถรองรับการควบคุม Servo Motor และการเชื่อมต่อเครือข่าย WiFi ได้อย่างมีประสิทธิภาพ ทำให้สามารถพัฒนาระบบควบคุมแขนกลผ่าน Web Interface ได้โดยไม่จำเป็นต้องใช้อุปกรณ์เพิ่มเติม การควบคุมแขนกลผ่าน **Joystick** ช่วยให้ผู้ใช้งานสามารถควบคุมการเคลื่อนที่ของแขนกลได้อย่างสะดวกและตอบสนองต่อคำสั่งได้รวดเร็ว ในขณะที่การควบคุมผ่าน **Web Interface** ช่วยให้สามารถควบคุมแขนกลได้จากระยะไกลผ่านเครือข่าย WiFi

อย่างไรก็ตาม ระบบยังมีข้อจำกัดบางประการ เช่น ความแม่นยำของการเคลื่อนที่ของแขนกลที่ขึ้นอยู่กับความละเอียดของ Servo Motor และข้อจำกัดของแรงบิดของ Servo Motor ที่ใช้ในโครงงาน ซึ่งอาจส่งผลต่อการหยิบจับวัตถุที่มีน้ำหนักมาก

5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต

เพื่อให้ระบบแขนกลหุ่นยนต์มีประสิทธิภาพมากยิ่งขึ้น สามารถพัฒนาเพิ่มเติมได้ในอนาคต ดังนี้

1. เพิ่มจำนวน Servo Motor หรือปรับปรุงโครงสร้างแขนกลเพื่อเพิ่มความยืดหยุ่นในการเคลื่อนที่
2. เพิ่มเซนเซอร์ เช่น เซนเซอร์ตรวจจับวัตถุ หรือกล้อง เพื่อให้แขนกลสามารถทำงานแบบอัตโนมัติได้
3. พัฒนา Web Interface ให้มีฟังก์ชันการควบคุมที่หลากหลายมากขึ้น เช่น การควบคุมแบบ Slider หรือการแสดงสถานะของแขนกลแบบเรียลไทม์
4. พัฒนาระบบให้สามารถเชื่อมต่อกับอินเทอร์เน็ต (IoT) เพื่อควบคุมแขนกลจากระยะไกลผ่านเครือข่ายอินเทอร์เน็ต

บรรณานุกรม

1. Benitez, V. H. (2020). *Design of an affordable IoT open-source robot arm for robotics education*. IEEE Access.
2. Espressif Systems. (2023). *ESP32 Series Datasheet*. Espressif Systems. https://www.espressif.com
3. Kareemullah, H. (2021). *Design and implementation of robotic arm and control of moving via IoT with Arduino ESP32*. ResearchGate.
4. Thulasi, P. R., & Kumar, S. (2022). *Development of robotic arm control using Arduino controller*. International Journal of Engineering Research and Technology.
5. Telengre, H., & Patil, A. (2023). *ESP32 based robotic arm vehicle using IoT*. International Journal of Scientific Research and Development.
6. Monk, S. (2017). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.
7. Margolis, M. (2019). *Arduino Cookbook* (3rd ed.). O'Reilly Media.